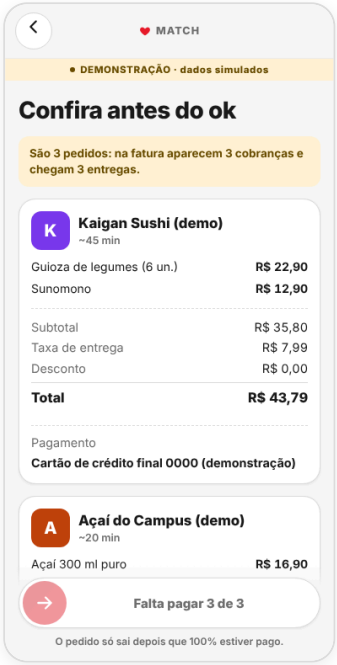
♥ match

Match — pedido em grupo com agentes no MCP do iFood

Hackathon SEMCOMP × iFood 2026 · "Vendendo com Agentes" · São Carlos







Entregáveis	01 a 06 do regulamento
Equipe	Thales Miguel Hajes · Milan · Zenon · Antonio
Data	18 de outubro de 2026
Repositório	github.com/scorpion-bits/Hackaton-Ifood

Protótipo de hackathon · não afiliado ao iFood. Dados simulados aparecem com o selo DEMONSTRAÇÃO.

Sumário

01 **Proposta de Negócio** p. 3

Problema, público, solução, valor para o iFood, modelo de negócio, métricas e viabilidade.

02 **Desenho de Solução** p. 8

Divisão IA × código × MCP, componentes, sequência completa de um Match em grupo e fluxo do motor.

03 **Racional da Experiência** p. 13

Princípios, jornadas do organizador e do convidado, leilão e deck, confiança, acessibilidade e estados.

04 **Interface Visual** p. 18

Linguagem visual, tokens e galeria com as capturas reais das telas do protótipo.

05 **Documento Técnico Resumido** p. 22

Leilão de agentes, otimizador, acerto em centavos, estados da sala, segurança, uso do MCP, limites e testes.

06 **Feedback sobre os MCPs** p. 30

Top 3 problemas por gravidade, o que funcionou bem e o registro de dores durante o evento.

Pitch: pitch/pitch.pdf · **Avaliar em 5 minutos:** README.md · **Índice para a banca:** docs/ENTREGA.md

Os textos deste PDF são gerados dos arquivos Markdown do repositório (docs/entrega/gerar_pdf.mjs). Em caso de diferença, vale o repositório.

01

Proposta de Negócio

Fonte no repositório: docs/proposta-negocio.md

> Entregável 1 do Hackathon SEMCOMP × iFood "Vendendo com Agentes". Leitura de 5 minutos. Protótipo de hackathon, não afiliado ao iFood. Todo número tem fonte com link (acesso em 17/10/2026, salvo indicação). O que é **hipótese** está escrito assim e diz como validar. Fontes detalhadas: `docs/pesquisa-mercado.md`.

1. Resumo

1. **Problema:** pedir comida em grupo exige que alguém vire o "gerente do pedido": pergunta o que cada um quer, acha lojas que agradem todo mundo, cabe no bolso de cada um, monta o carrinho e depois cobra a vaquinha.
2. **Solução:** no **Match**, cada pessoa escreve uma frase ("japonês até R$ 45", "sem carne"); um **agente vendedor por restaurante**, que só enxerga o próprio cardápio, disputa o pedido; o **IBits**, agente comprador da turma, escolhe a melhor combinação (1 ou mais lojas) com justiça entre as pessoas, tudo sobre o **MCP do iFood**. Nada sai sem o ok do organizador.
3. **Por que agora:** comércio feito por agentes já está em produção. O Google lançou o protocolo de pagamentos para agentes AP2, que funciona sobre o MCP (Google, 16/09/2025); a Visa lançou o Intelligent Commerce (Visa, 30/04/2025); a Swiggy, na Índia, recebe pedidos de comida por MCP desde janeiro (Swiggy, 27/01/2026); e o próprio iFood fecha **um** pedido em cerca de 30 segundos no Ailo com o One-Click Buy (Canaltech, 16/07/2026). Falta o pedido **da turma**.

2. Problema e público

Ocasiões: turma da faculdade depois da aula, república na sexta à noite, almoço do time no trabalho, jantar de família no domingo, jogo do Brasil com os amigos.

Quem	Dor hoje
Organizador (quem tem o app e o cartão)	Faz a logística sozinho no grupo de WhatsApp, adianta o dinheiro e depois cobra Pix de cada um
Convidados	Fila de mensagens, "tanto faz", alguém cede o restaurante que queria ou fica com o que não queria; quem tem restrição (vegetariano, sem lactose) sobra

Não existe número público brasileiro sobre pedido de delivery em grupo (frequência, tempo para decidir, desistência). Procuramos e não achamos (ver `docs/pesquisa-mercado.md` §4). Por isso fizemos nossa própria pesquisa.

> **Pesquisa da equipe, SEMCOMP São Carlos, 17–18/10/2026, n = [preencher], n que já pediu em grupo = [preencher].** Amostra de conveniência (participantes de um evento de computação), anônima, não representativa.
>
> - Pediram delivery em grupo no último mês: [preencher]%
> - Levaram **mais de 15 min** para decidir: [preencher]%
> - **Cada um pediu separado** no próprio app: [preencher]%
> - Tiveram **atrito** (alguém desistiu, ficou com o que não queria ou o grupo desistiu): [preencher]%
> - Alguém **queria outro restaurante**: [preencher]%
> - Alguém **adiantou e cobrou Pix depois** (ou ficou faltando dinheiro): [preencher]%
> - Intenção de uso do Match: média [preencher] de 5; notas 4 ou 5: [preencher]%
> - Principais objeções: [preencher]

> (Se n < 20, trocar porcentagens por "X de Y pessoas".)

Apoio de fora (outros países, encomendados, não são do Brasil): no Reino Unido, quase metade dos entrevistados diz que escolher onde comer em grupo leva em média 25 minutos (OnePoll para Meerkat Meals, 2018, amostra não informada; Public Sector Catering). Nos Emirados, 43% dizem gastar perto de 20 minutos decidindo o que pedir online (YouGov para Deliveroo, 2022; Deliveroo UAE).

3. Solução e diferenciais

Como funciona em 5 passos: (1) o organizador cria a sala no iFood e mostra o QR; (2) cada convidado entra **sem login** e escreve uma frase em até 30 s; (3) os agentes das lojas fazem seus lances, com preço e prazo **vindos do MCP**; (4) o IBits monta até 3 combinações e o grupo dá "Match"; (5) pagamento **junto** ou **separado** (vaquinha: Pix direto para a chave do organizador; o pedido só sai com **100% pago**), conferência do carrinho real e "deslize para enviar".

Pedido em grupo já existe lá fora e no Brasil. Não vendemos isso como inédito. O que é novo é a **composição por IA** para N pessoas em N lojas.

	Rappi (inclui Brasil)	Uber Eats	DoorDash	Ailo (iFood)	Match
Pedido em grupo por link	sim, criador paga (Supervarejo, 11/2023; só vimos o resumo da matéria)	sim, 1 restaurante, com divisão da conta (TechCrunch, 03/2022)	sim, com rateio de taxas (DoorDash)	não citado pelo iFood (iFood)	sim, convidado sem login
Mais de 1 loja	não encontrado	2 lojas próximas, 1 taxa (TechCrunch, 10/2023)	DoubleDash, 2ª loja após o checkout (DoorDash)	não	sim, até 3 lojas (N pedidos, N taxas, avisado antes)
Quem escolhe os itens	cada pessoa, à mão	cada pessoa, à mão	cada pessoa, à mão	IA, para **1 pessoa**	IA, para **o grupo**, com justiça
Frase livre ("algo leve até R$ 30")	não	não	não	sim (1 pessoa)	sim, por pessoa
Lojas disputando o pedido	não	não	não	não	sim, um agente por loja
Trava "só sai com 100% pago"	não encontrado	não encontrado	não encontrado	—	sim

Diferenciais em uma linha cada:

- **Composição por IA:** N pessoas × N lojas × orçamentos e restrições diferentes. O otimizador puxa para cima quem ficou pior (ninguém fica mal atendido para os outros ganharem).
- **Lojas disputando:** cada restaurante tem um agente vendedor. Não encontramos iniciativa pública de agente de restaurante negociando com agente do comprador em delivery (não dizemos "ninguém faz").
- **Vaquinha com trava:** o pedido só é enviado quando todos pagaram; prazo de 10 min + 2 min por pessoa acima de 2.
- **Números só do iFood:** preço, taxa e prazo vêm das tools do MCP. Fala de agente com número inventado é trocada por texto pronto.
- **Nada sai sem ok:** `submitOrder` só depois de confirmação humana explícita.

4. Valor para o iFood

(a) Pedido de grupo que hoje não acontece ou vira pedidos picados. *Hipótese a validar:* quando o grupo não se entende, ou ninguém pede, ou cada um pede separado (com N taxas e N entregas), ou o grupo vai comer fora. O Match transforma isso em um pedido organizado. Validação: indicadores de atrito e "cada um pediu

separado" da nossa pesquisa (§2) e o piloto (§5).

(b) Mais lojas por sessão. Um grupo com vontades diferentes pode fechar em 2 ou 3 lojas em vez de alguém ceder. *Hipótese:* mais lojas atendidas por ocasião, sobretudo lojas de nicho (vegana, sem glúten, sobremesa).

(c) Nova linha de receita: "Patrocinado" no canal de agentes. O modelo é comprovado: anúncio de loja dentro do app de delivery passa de US$ 1 bilhão por ano na Uber (Digiday, 08/05/2025) e na DoorDash (transcrição da fala de resultados, 06/08/2025). O iFood já tem o **iFood Ads**: leilão em que a loja define um lance **por visita** ao cardápio, e a posição depende também de avaliação e indicadores operacionais. Frase do próprio iFood: **"Dinheiro sozinho não compra posição."** (Blog iFood Parceiros, 18/09/2026). E o iFood quer **multiplicar por 5** a frente de Ads até 2030 (Startups, 12/11/2025).

O Patrocinado leva essa regra ao extremo, no momento em que um **agente** compra:

- A loja patrocinada tem o lance **mostrado primeiro**, com o rótulo "Patrocinado".
- O pagamento **nunca** muda nota, filtro ou escolha do IBits e não garante vaga entre as lojas consultadas. Na demo, a patrocinada fala primeiro e **perde**.
- **O iFood vende atenção, não a decisão.** O agente comprador trabalha para o grupo; se ele pudesse ser comprado, ninguém confiaria nele, e o canal inteiro perderia valor.
- Rótulo sempre visível, em texto (não só cor), cumprindo o princípio da identificação da publicidade do CDC (Lei 8.078/1990, art. 36: "A publicidade deve ser veiculada de tal forma que o consumidor, fácil e imediatamente, a identifique como tal"; site do Planalto fora do ar em 18/10, texto conferido em Meu Site Jurídico).
- No protótipo, o Patrocinado é **demonstração**; no modo real nenhuma loja é patrocinada.

(d) Uma capacidade que o Ailo pode usar. O Ailo já entende pedido por conversa e tem 48% mais chance de a busca virar pedido, segundo o iFood (iFood, 10/10/2025). O Match não concorre com ele: é a peça de **grupo** (vários desejos, várias lojas, justiça, vaquinha) que pode entrar no Ailo e no app, ao lado do pagamento agêntico que o iFood já lançou (Mobile Time, 28/08/2026).

(e) Dados de preferência de grupo, com consentimento. Que combinações de lojas os grupos aceitam, que restrições aparecem juntas, quanto cada perfil aceita gastar. Só agregado e com consentimento explícito; o convidado sem login guarda as próprias preferências no aparelho dele.

Escala do iFood (para dimensionar, não é projeção nossa): mais de 180 milhões de pedidos por mês, 65 milhões de clientes e mais de 500 mil estabelecimentos (Portal de Dados iFood). Não temos dado de quantos desses pedidos são de grupo; não estimamos.

5. Modelo de negócio e métricas

Como o Patrocinado cobraria (proposta, sem preço):

- Mesma lógica do iFood Ads: a loja define orçamento diário e um **lance por aparição no leilão** do Match (equivalente ao lance por visita de hoje). Paga só quando o lance dela é mostrado a um grupo.
- Ordem de exibição entre patrocinadas = lance × qualidade (avaliação, cancelamento, tempo de entrega), como no iFood Ads.
- Entra na **Central de Crescimento** do Portal do Parceiro, junto do iFood Ads de hoje.
- Preço: definido pelo iFood. **Não temos e não inventamos** valor; o iFood também não divulga receita de Ads.

Receita principal continua a mesma: comissão e taxa de cada pedido. Um Match com 3 lojas gera 3 pedidos.

Métricas de sucesso

Métrica	O que mostra
% de salas que viram pedido	o Match tira o grupo da indecisão?
Tempo da sala criada até o pedido enviado	o organizador economiza trabalho?
Itens por pedido de grupo e pessoas por sala	o pedido de grupo é maior que a soma dos picados?

Métrica	O que mostra
Lojas por sessão	mais lojas atendidas por ocasião?
% de vaquinhas que fecham no prazo	a trava de 100% funciona sem irritar?
CTR do Patrocinado (abriu a loja depois de ver o lance)	a atenção vendida tem valor para a loja?
% de grupos que dão match no 1º cartão	a IA acerta?
NPS do organizador e do convidado, separados	os dois lados saem satisfeitos?

Piloto proposto: 4 semanas numa cidade universitária como São Carlos (muitas repúblicas e turmas).

- Grupo A: entrada "Pedir em grupo? Dê um Match" na home. Grupo B: app como hoje.
- Semana 1–2 sem Patrocinado; semana 3–4 com Patrocinado para lojas voluntárias, para medir se o rótulo muda a confiança (NPS e % de salas que viram pedido não podem cair).
- Critério de sucesso: definido com o iFood antes de começar (não chutamos número).

6. Viabilidade

O que já funciona (protótipo): o fluxo inteiro em modo demonstração (sala, frases livres, leilão, deck, vaquinha com Pix "copia e cola" real, conferência e envio com consentimento); cliente do MCP real com OAuth PKCE seguindo a sequência da documentação (`listAddresses` → busca → carrinho → `getCart`); [preencher: até onde o MCP real rodou no PC da equipe e latência medida no rastro]. Detalhes em `docs/tecnico.md` .

O que falta para produção

Falta	Caminho
Agente da loja feito pelo próprio restaurante	Hoje simulamos o agente com o cardápio público do MCP. Em produção, a loja pluga a própria política (promoção, combo) pelo Portal do Parceiro / Merchant API; o iFood controla as regras (validação, rótulo, sem número inventado)
Vaquinha sem Pix manual	**iFood Pago** recebendo cada parte (nosso app não pode guardar dinheiro de terceiros; intermediar pagamento exige instituição autorizada). O iFood Pago já existe (Prosus FY2026) e o iFood testa dividir o pagamento (Tecnoblog)
1 entrega para lojas próximas	Logística interna, como Uber Eats e DoorDash já fazem (§3). Pelo MCP não dá: hoje N lojas = N entregas, e o app avisa
Antifraude da vaquinha	Confirmação automática do pagamento (no lugar do "recebi" do organizador), limite por sala, detecção de salas falsas
Grupos grandes	Hoje até 6 pessoas por sala (limite do otimizador); para 15+, poda por loja e votação no deck

Riscos e mitigação

Risco	Mitigação
Alergia: o cardápio nem sempre informa	Restrição vira "segundo a descrição" + aviso "pode não estar informado, confirme com a loja"
Patrocinado minar a confiança	Só ordem de exibição, nunca a escolha; rótulo em texto; teste no piloto (semana 3–4)
Custo de LLM com vários agentes por Match	Um agente por loja, em paralelo, com tempo máximo; **fallback determinístico** sem LLM (a demo inteira roda sem chave); custo medido: [preencher, se medido no ensaio]
Limite de chamadas (rate limit, erro 429)	Fallback cobre o erro; produção: cache de cardápio por loja e lote de chamadas
IA inventar preço	A IA não escreve número que não veio de uma tool; fala com número fora do lance cai no texto pronto
Pedido enviado sem querer	Só com confirmação humana; `getCart` de novo antes de enviar; se o total mudou, pede novo ok

7. Visão

Além do Brasil. O iFood hoje opera só no Brasil (saiu da Colômbia em 11/2022, Baguete), mas o grupo Prosus concluiu a compra da Just Eat Takeaway em 10/2025 (RetailDetail). *Hipótese:* o Match é uma camada que funciona em qualquer marketplace do grupo, porque entra texto em qualquer idioma e sai uma cesta com pagamento local plugável. Validação: a mesma engine com cardápio simulado em inglês e espanhol.

Além de comida. O iFood já vende mercado, farmácia, pet e bebidas (iFood, 04/06/2026), e, em relato oral à equipe durante o evento (não documentado), os mentores disseram que o catálogo de mercado vem com preço no MCP (`docs/planos/etapa-01b-match-estrategia.md` §12). A mesma lógica serve para a compra do mês da república (cada um diz o que precisa, a IA divide em lojas e a vaquinha fecha) ou a farmácia da família.

A frase: no canal de agentes, o iFood pode vender **atenção** para as lojas sem nunca vender **a decisão** do cliente. O Match mostra que dá para ter as duas coisas no mesmo pedido.

02

Desenho de Solução

Fonte no repositório: docs/tecnico.md §1–4

> Entregáveis 2 (desenho de solução) e 5 (documentação técnica) do Hackathon SEMCOMP 29 × iFood. Protótipo de hackathon, não afiliado ao iFood. Como rodar: `README.md` . Estado do código em 18/10/2026 (madrugada). O que ainda depende de dado externo vem marcado **[pendente]**.

1. Visão geral

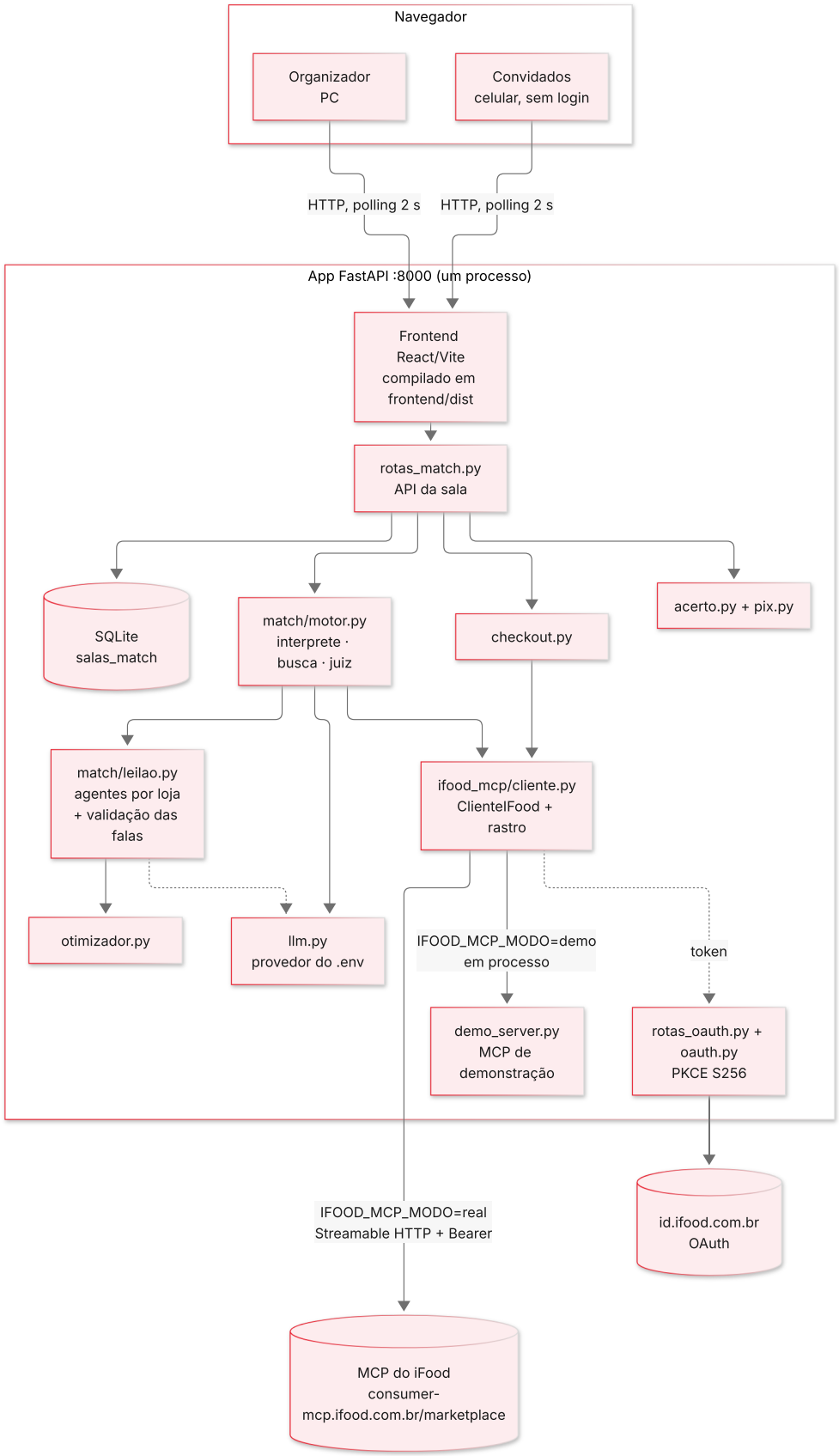
O Match transforma frases livres de 1 ou N pessoas em um pedido viável no iFood: itens reais, preço real, loja(s) real(is), custo total e motivo de cada escolha. Só o organizador entra no iFood; os convidados entram por link/QR, sem login.

Divisão de trabalho entre IA e código (decisão central do projeto):

Quem	Faz
IA (LLM, saída JSON)	entende a frase (slots, restrições, orçamento, perfil); dá nota 0–1 e motivo a cada item; o agente de cada loja escolhe o que oferecer a quem e escreve a fala
Código determinístico	filtra restrições e orçamento; valida cada oferta e cada fala dos agentes (fallback por template); decide o agrupamento por loja; enumera e pontua combinações; divide a conta em centavos; gera o Pix; segue a ordem do checkout; exige consentimento
MCP do iFood	**única fonte** de preço, taxa, prazo, disponibilidade, carrinho e pedido

Toda chamada de IA tem um caminho determinístico de reserva. Com `LLM_PROVIDER=mock` a demo inteira roda sem chave e sem internet.

2. Componentes

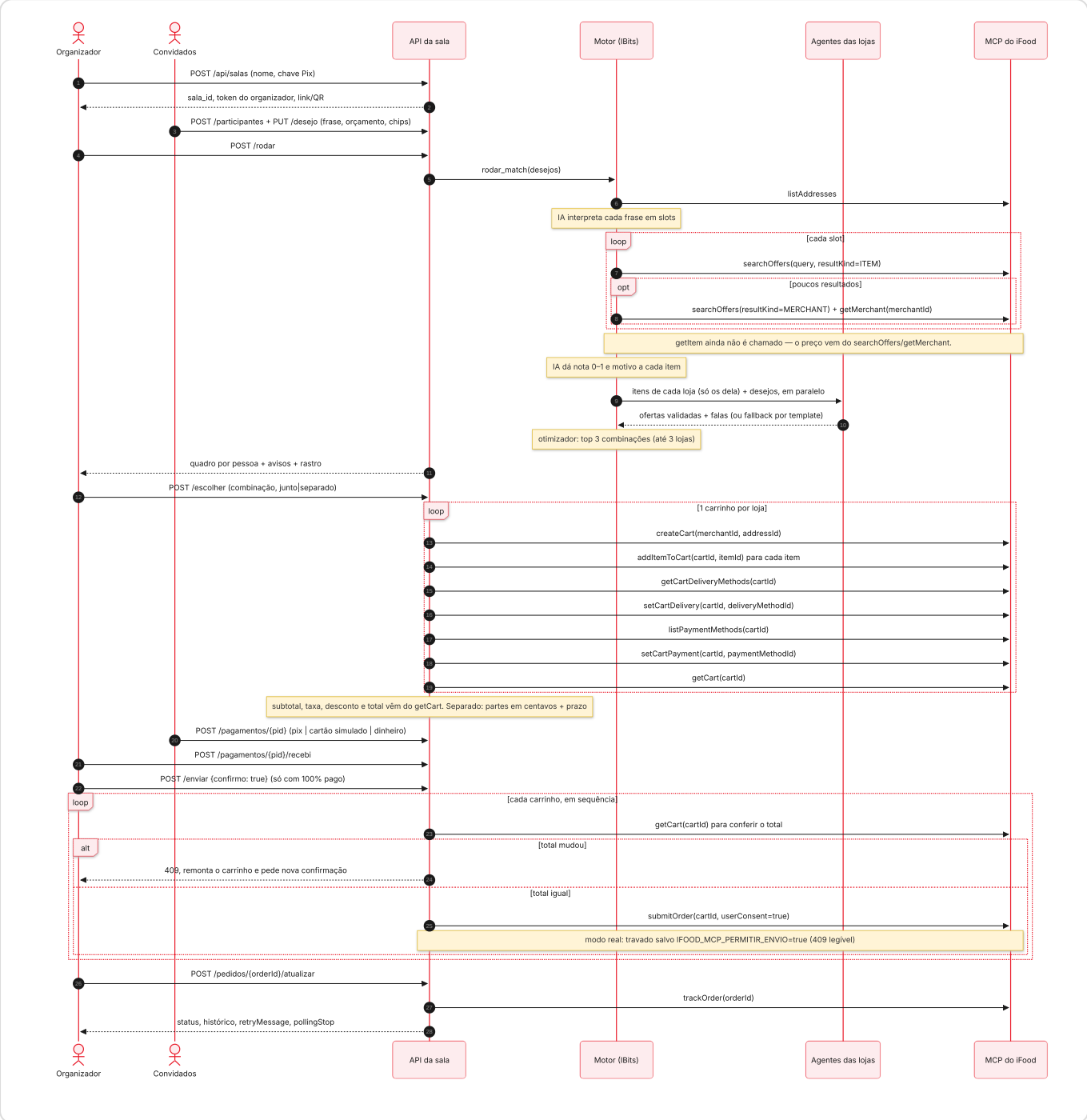


Componente	Arquivo	Papel
Frontend	`frontend/src/`	React 19 + TypeScript (Vite). Rotas por hash: `#/` (início estilo app), `#/s/{id}` (convidado), `#/s/{id}/org` (organizador), `#/conectado` (volta do login). Polling de 2 s em `GET /api/salas/{id}` . Compilado e servido pelo próprio FastAPI (uma URL na demo). Telas do Match: `telas/Leilao.tsx` (disputa) e `telas/Deck.tsx` (deck de combinações), com peças em `componentes/leilao/` (seção 5).
API da sala	`backend/app/rotas_match.py`	Criar sala, entrar, desejo, rodar, escolher, pagar, enviar, acompanhar. Ações do organizador exigem o header `X-Organizador` .
Sala	`backend/app/match/salas.py`	Estado inteiro da sala em JSON numa tabela SQLite própria. Escrita atômica (trava + transação). Chamadas ao MCP ficam fora da trava de gravação e o estado é revalidado depois.
Motor	`match/motor.py`	`listAddresses` → interpretar (+ quantidades) → buscar → julgar → leilão → otimizar → fechar o leilão → avisos. `MATCH_LEILAO=off` no `.env` pula o leilão (o otimizador volta a ver todos os candidatos).
Intérprete	`match/interprete.py`	Frase → 1 ou mais **slots** ("japonês e um sorvete" = 2 slots). IA com validação; reserva por palavras-chave + chips. **Consulta livre sem IA:** item fora das categorias conhecidas ("coxinha", "feijoada completa") vira consulta pelo próprio texto (até 4 palavras, sem orçamento, restrição nem palavras genéricas; "coxinha ou esfiha" = alternativas do mesmo slot). **Quantidades:** "2 sorvetes"/"duas coxinhas" viram slots iguais (até 3 iguais e 4 slots por pessoa; "pizza 4 queijos" não é quantidade); o que não couber vira aviso, nunca some calado.
Busca	`match/busca.py`	Único arquivo que lê campos das tools de descoberta. Preço só da tool; item sem preço é descartado. ≤ 8 candidatos por slot, diversificados por loja. **Leitura robusta de endereço:** acha a lista do `listAddresses` em formatos variados (raiz lista, `addresses` , `data.addresses` ...), prefere o último usado/padrão e lê coordenadas com nomes alternativos (`lat` / `lng` / `lon`).
Juiz	`match/juiz.py`	Nota 0–1 + motivo curto. Restrição sem tag no item não elimina: o motivo avisa "confirme com a loja".
Leilão	`match/leilao.py` + `match/politica_lojas.json`	1 agente vendedor por loja, validação dura das ofertas e falas, fallback por template, medições (seção 5).
Otimizador	`match/otimizador.py`	Código puro, sem I/O nem IA (seção 6).
Checkout	`match/checkout.py`	Único arquivo que lê campos de carrinho/pedido. 1 carrinho por loja, envio com consentimento, `trackOrder` .
Acerto e Pix	`match/acerto.py` , `match/pix.py`	Partes por pessoa em centavos; Pix copia e cola (BR Code estático).
Cliente MCP	`ifood_mcp/cliente.py`	Mesma interface nos dois modos; registra cada chamada no **rastro** (tool, resumo sem dados sensíveis, ms, ok/erro). No modo real lê o `input_schema` de cada tool (1 `list_tools` por sessão) e adapta os argumentos: `latitude` / `longitude` viram `userLocation: {latitude, longitude}` quando a tool declara esse campo, `query` vira o campo de texto declarado (`term` , `q` ...) e argumentos não declarados são descartados; o rastro mostra o que foi descartado ("descartados: latitude, longitude"). **Trava de envio:** no modo real, `submitOrder` só é chamado com `IFOOD_MCP_PERMITIR_ENVIO=true` (seção 9).
MCP de demonstração	`ifood_mcp/demo_server.py`	Servidor MCP (SDK `mcp` 2.x) com 17 tools de **mesmo nome e campos** da doc oficial; lojas fictícias; todo payload com `"demonstracao": true` .
OAuth	`ifood_mcp/oauth.py` , `rotas_oauth.py`	Authorization Code + PKCE S256 + client secret; tokens em arquivo fora do git; renovação automática.

Componente	Arquivo	Papel
Diagnóstico	`ifood_mcp/diagnostico.py`	Roda no PC da equipe: `list_tools` + chamadas de leitura → amostras **sanitizadas** em `docs/mcp-real/`. Nunca chama `submitOrder`.

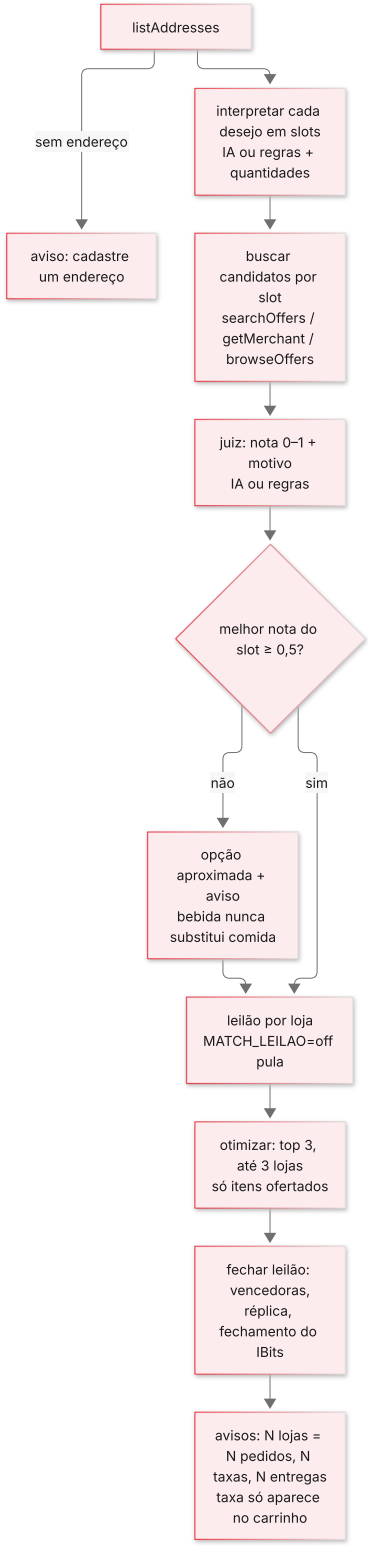
3. Sequência completa de um Match em grupo

Ordem obrigatória da documentação do MCP: `listAddresses` → `searchOffers` → `getMerchant` → `getItem` → `createCart` → `addItemToCart` → `getCartDeliveryMethods` → `setCartDelivery` → `listPaymentMethods` → `setCartPayment` → `getCart` → `submitOrder({cartId, userConsent: true})` → `trackOrder`.



Cada seta para o MCP vira uma linha do rastro na tela, com o nome da tool da documentação. O resumo do rastro não mostra latitude, longitude, endereço, e-mail, telefone nem CPF.

4. Fluxo do motor



Regras de relevância já no código: item com nota < 0,25 sai se houver opção melhor; slot cujo melhor item fica abaixo de 0,5 vira "Opção aproximada: ..." com aviso "Não achei X; a opção mais próxima é Y"; se ninguém tem item relevante, não há combinação e a API devolve 409 com a pergunta "Quer mudar o pedido?".

03

Racional da Experiência

Fonte no repositório: docs/racional-ux.md

> Entregável 3 do hackathon SEMCOMP × iFood. Leitura de 3 minutos. Protótipo de hackathon, não afiliado ao iFood. Dados da demo são simulados e levam o selo DEMONSTRAÇÃO.

1. O problema, em uma frase

Pedir comida em grupo hoje exige que alguém vire o "gerente do pedido": perguntar o que cada um quer, achar lojas que agradem todo mundo, caber no bolso de cada um, montar o carrinho e depois cobrar a vaquinha.

Quem	O que sente hoje	O que o Match entrega
Organizador (o único com conta no iFood)	Trabalho de logística e de cobrança, sozinho	Cria a sala, toca em "Rodar Match", confere e dá o ok
Convidados (sem login)	Fila de mensagens, "tanto faz", ficam sem o que queriam	Escrevem uma frase (meta de ≤ 30 s, não medida) e recebem o prato com o motivo

Insight central: ninguém quer ser o gerente do pedido. Então o trabalho chato vai para a IA e para os agentes das lojas, e as pessoas só fazem o que só elas podem fazer: dizer o que querem e dar o ok final.

2. Princípios de design

1. **Dentro do app do iFood.** O Match é uma entrada na home ("Pedir em grupo? Dê um Match"), não um app novo. Familiaridade reduz o que a pessoa precisa aprender.
2. **Um só login.** Só o organizador entra no iFood. Convidado entra por link ou QR, sem cadastro.
3. **30 segundos para o convidado (meta, não medida).** Nome, uma frase livre e chips opcionais (sem glúten, sem lactose...). Nada mais.
4. **Escreva do seu jeito.** Frase livre ("japonês e um sorvete, até 45") em vez de formulário. A IA entende preço, pressa, restrição e vontade.
5. **Todo número é explicado e vem do iFood.** Preço, taxa e prazo vêm das tools do MCP. A IA nunca escreve um número que não veio de uma tool; se tentar, a fala é trocada por um texto pronto.
6. **Nada sai sem o seu ok.** O pedido só é enviado depois de uma confirmação humana explícita.
7. **Honestidade.** O que é simulado tem selo. O que o cardápio não garante (alergia) é dito com todas as letras. O que é anúncio tem rótulo.
8. **Sem taxa escondida.** Se o grupo ficar em 3 lojas, a tela diz: 3 pedidos, 3 taxas, 3 entregas.

3. Jornadas

Organizador

Passo	Objetivo	Decisão de design	Por quê
Home → banner "Dê um Match"	Começar sem procurar	Banner na home e atalho na barra inferior (selo "protótipo"), no visual do app	Quem já usa o iFood acha sem aprender nada
"Quem vai comer?" (só eu / em grupo)	Mesmo fluxo para 1 ou N	O individual é o grupo de 1, sem sala	Um fluxo só para aprender e manter

Passo	Objetivo	Decisão de design	Por quê
Sala com QR e link	Chamar a turma	QR grande no projetor/tela, link para WhatsApp, lista "quem entrou" e "respondeu"	Padrão já conhecido de Uber Eats e DoorDash (ver `design-ifood.md` §4)
"Rodar Match"	Sair da espera	Liberado com pelo menos 1 resposta; mensagem "A IA está lendo os desejos e consultando as lojas"	Espera com explicação parece trabalho, não travamento
Leilão ao vivo	Ver como a escolha foi feita	Lojas disputam, falam e mostram preço e afinidade; botão **Pular**	Transparência (ver §4)
Deck "É um Match!"	Escolher entre até 3 opções	1 cartão por combinação, deslizar ou botões	Decidir sem tabela (ver §4)
Junto ou separado	Definir quem paga	Duas opções com a consequência escrita ("na fatura aparecem N cobranças")	A pessoa escolhe sabendo o efeito
Vaquinha	Receber de todos	Barra "x de N pagos", contador de prazo, botão "Recebi" para Pix e dinheiro	O organizador vê quem falta sem cobrar no grupo
"Confira antes do ok"	Conferir tudo	Subtotal por loja, taxa do carrinho, total geral, aviso de N entregas	O último momento de corrigir
"Deslize para enviar"	Dar o consentimento	Gesto deliberado (ou botão "Confirmar envio")	Evita envio por toque acidental
Pedidos	Acompanhar	1 cartão por pedido, status individual e rastro das tools MCP	Cada loja entrega no seu tempo

Convidado

Passo	Objetivo	Decisão de design	Por quê
Lê o QR	Entrar	Abre no navegador, sem app e sem login	Barreira zero
"O que você quer?"	Dizer a vontade	Nome + frase livre + chips de 1 toque + orçamento opcional; "Lembrar de mim neste aparelho"	Meta de caber em 30 s (não medida); a preferência fica no celular dele
Espera	Saber o que acontece	"Aguarde o organizador rodar o Match"; pode mudar o desejo	Sem ansiedade, sem refazer
Leilão e deck	Ver o seu prato e o motivo	Mesmo leilão do organizador; deck só leitura	Todos veem a mesma coisa; só um decide
Sua parte	Pagar a sua parte	Valor por pessoa (itens + rateio da taxa só das lojas dele); Pix, cartão ou dinheiro	Cada um paga só o que consumiu

4. Por que o leilão e por que o deck

Leilão ao vivo

- **Torna visível o trabalho da IA.** Cada restaurante tem um agente vendedor que só enxerga o próprio cardápio (vindo do MCP) e oferece itens a cada pessoa. O IBits, agente comprador do grupo, escolhe entre os lances. Sem o leilão, isso seria uma caixa-preta de alguns segundos.
- **Confiança por transparência.** Cada lance mostra preço e afinidade. Loja que não atende diz "Passo". O fechamento explica quem venceu e avisa que as taxas aparecem no carrinho, antes do ok.
- **Números honestos.** A fala da loja só pode citar números do próprio lance (preço, prazo, avaliação, contagens). Fala com número inventado (também por extenso), contagem errada, "todo mundo" sem atender todos, restrição que a tag do item não confirma, promessa (frete, cupom...) ou superlativo cai no texto pronto, e a tela mostra a origem ("agente da loja (IA)" ou "texto conferido").
- **Patrocinado rotulado e sem poder de compra.** A loja patrocinada aparece primeiro e com selo "Patrocinado", mas nunca muda nota, filtro ou escolha, e não garante vaga entre as lojas consultadas. Na

demo, a patrocinada **não** vence: mostra que pagar não compra o pedido. No modo real, nenhuma loja é patrocinada.
- **Respeita o tempo.** Uma fala por vez (~0,8 s), botão **Pular** sempre visível, e o aparelho só vê o leilão uma vez por sala.

Deck

- **Decidir entre 3 opções sem tabela.** Cada cartão é uma combinação inteira: afinidade %, "N de M bem atendidos", lojas e, por pessoa, item · loja · preço · motivo.
- **Gesto conhecido.** Deslizar para a direita = Match; para a esquerda = próxima. O padrão já é familiar de apps de "match".
- **Botões equivalentes.** Os mesmos dois botões ficam sempre na tela. Ninguém depende do gesto (ver §7).
- **"É um Match!"** fecha a decisão com um momento de alegria, e leva direto a "junto ou separado".

5. Confiança e segurança

Risco	Decisão de design
Pedido enviado sem querer	`submitOrder` só com `userConsent: true`, depois de "Deslize para enviar" ou "Confirmar envio" na tela
Total diferente do mostrado	Antes de enviar, o carrinho é lido de novo (`getCart`). Se o total mudou, as partes são recalculadas e o app pede novo ok
Confundir demo com pedido real	Selo DEMONSTRAÇÃO fixo no topo; cartão simulado com selo; "Simulado: nenhuma cobrança real"
Alergia	O cardápio nem sempre informa. Restrição vira "segundo a descrição" e aparece o aviso "pode não estar informado, confirme com a loja"
Dinheiro de terceiros	O app não guarda dinheiro. Pix vai direto para a chave do organizador; a chave fica só no aparelho dele
Vaquinha sem fim	Prazo de 10 min + 2 min por pessoa acima de 2, com contador visível. O pedido só sai com 100% pago
Falha no meio do envio	O envio para na primeira falha; o pedido que já saiu continua valendo e a tela diz o que fazer

6. Identidade visual

- **Alinhada ao app do iFood**, para o Match parecer uma função do app e não um corpo estranho: vermelho `#EA1D2C` (o único hex confirmado em fonte do iFood), fundo claro, cartões brancos com cantos arredondados, home com "Entregar em", verticais e cards de loja. Tokens e fontes estão em `docs/design-ifood.md` §5.
- **Sem logo, sem fonte proprietária, sem fotos do iFood.** Usamos Inter, emoji neutros por categoria e o nome "Match". Por quê: não temos licença para os ativos da marca, e um protótipo não deve se passar pelo app oficial. Rodapé fixo: "Protótipo de hackathon · não afiliado ao iFood".
- O resto da paleta (vermelho escuro, cinzas, verde) é escolha nossa e está marcado como não oficial no `design-ifood.md`.

7. Acessibilidade

- **Contraste (calculado pela fórmula da WCAG 2.x com os tokens do `estilo.css`):** texto `#1A1A1A` no branco = 17,4:1; texto suave `#6F6F6F` no branco = 5,0:1; aviso `#6B4E00` no fundo `#FFF4D6` = 7,1:1. O vermelho `#EA1D2C` no branco dá 4,46:1 (um pouco abaixo de 4,5:1 para texto pequeno) → **regra:** texto pequeno em vermelho usa `--marca-escura` `#C8101E` (5,9:1), como já fazem erros, chips ativos e o botão do banner; o vermelho puro fica em fundos de botão e texto grande em negrito. Pendente: links (`a`) ainda usam o vermelho puro. O verde `--ok` `#2F8A55` dá 4,3:1 → sempre em negrito e junto de palavra ou ícone ("pago", "respondeu"), nunca como única pista.

- **Cor nunca é a única pista.** Status tem palavra ("⏳ aguardando", "pago"); Patrocinado tem texto, não só cor.
- **Alvos de toque grandes** nos botões principais e chips; botões ocupam a largura no celular.
- **Alternativa a todo gesto.** Deck: botões "próxima" e "dar match". Envio: botão "Confirmar envio" além do deslize.
- **Movimento reduzido.** Com `prefers-reduced-motion`, o leilão mostra as falas sem animação e o deck troca de cartão sem voo (o protótipo aprovado já trata isso; as telas do app seguem a mesma regra). **Pular** existe para todos.
- **Leitores de tela.** Chips com `aria-pressed` e grupo rotulado (já no app), linha do tempo do pedido rotulada, ícones decorativos com `aria-hidden`; no leilão, o feed de falas é região viva (`aria-live`), como no protótipo.
- **Linguagem simples.** Frases curtas, voz ativa, sem jargão: "Nada é enviado ao iFood ainda: você revisa antes."

8. Erros e estados

Situação	O que a pessoa vê
Não achei o que a pessoa pediu	Opção aproximada com o motivo ("É o mais perto do que você pediu agora") em vez de vazio
Nenhuma combinação coube	"As restrições e orçamentos deste grupo não encontraram opções" + "Ajuste o orçamento ou o que cada um quer e rode de novo"
Loja não atende ninguém	No leilão, a loja diz "Passo"; ela some da escolha sem erro
IA lenta ou fora	Fallback determinístico em poucos segundos; o leilão toca com falas prontas
Prazo da vaquinha acabou	"O prazo acabou e Fulano não pagou" → **Organizador cobre** (assume a parte) ou **Tirar os itens dessa pessoa** (recalcula)
Total mudou no carrinho	O app para, mostra o novo total e as novas partes e pede novo ok
Loja recusou o pedido	"O iFood recusou este pedido" → o Match para, mostra o que saiu e o que não saiu e avisa quem já pagou; o acerto da devolução ainda é feito pelo organizador (no protótipo não há recálculo automático)
Carregando	Mensagem do que está acontecendo ("Montando carrinhos..."), nunca tela parada

9. O que mediríamos em produção

Ainda **não temos dados de uso**. Não citamos números de pesquisa. Estas são as métricas e o teste que faríamos.

Métrica	Pergunta que responde
Tempo da sala criada até o pedido enviado	O Match tira mesmo o trabalho do organizador?
Tempo do convidado até enviar o desejo	A promessa de 30 s se cumpre?
% de salas que viram pedido	Onde o grupo desiste (espera, deck, vaquinha, envio)?
% de grupos que dão match no 1º cartão	A primeira combinação é boa o bastante?
% de "Pular" no leilão	O leilão ajuda ou atrapalha?
% de vaquinhas que fecham no prazo, e quantas vezes o organizador cobre	O prazo e a cobrança funcionam?
NPS do organizador e do convidado, em separado	Os dois lados saem satisfeitos?

Teste com usuários (proposta):

1. Grupos reais de amigos (3 a 4 pessoas), cada um no próprio celular, com a tarefa "peçam o jantar de hoje".
2. Comparar com o jeito atual (grupo de WhatsApp + um app de entrega): tempo total, mensagens trocadas e quem ficou satisfeito.
3. Observar em silêncio e perguntar no fim: "Em que momento você ficou em dúvida?" e "Você confiou no total?".
4. Variantes A/B: leilão ligado × direto ao deck; deck × lista simples.

No evento, a equipe aplica um formulário curto com participantes; o resultado, se coletado, fica em `docs/pesquisa-usuarios.md`, com o número de respostas.

04

Interface Visual

Fonte no repositório: docs/entrega/04-interface.md · docs/entrega/telas/

> Capturas reais do protótipo rodando no modo demonstração (celular, 390 × 780), tiradas nos smokes de 18/10 (B6; telas 5 a 9 refeitas no B9, quando o resultado passou a mostrar direto o pedido escolhido pelo IBits) com o fluxo completo passando pela interface. Os dados das lojas são simulados e aparecem com o selo **DEMONSTRAÇÃO**. Imagens em `docs/entrega/telas/` . Racional de cada escolha: entregável 03.

Linguagem visual

- **Alinhada ao app do iFood, sem se passar por ele:** vermelho `#EA1D2C` (único hex confirmado em fonte do iFood), fundo claro, cartões brancos com cantos arredondados, home com "Entregar em", verticais e cards de loja.
- **Sem logo, sem fonte proprietária, sem fotos do iFood:** fonte Inter, emoji neutros por categoria, wordmark próprio "match". Rodapé fixo: "Protótipo de hackathon · não afiliado ao iFood".
- **Mobile first:** convidados respondem pelo celular; o organizador pode usar o PC (QR grande na tela).

Token	Valor	Origem
Marca	`#EA1D2C`	confirmado (blog técnico do iFood)
Marca escura (texto pequeno em vermelho)	`#C8101E`	não oficial, escolhido por contraste 5,9:1
Texto / texto suave	`#1A1A1A` / `#6F6F6F`	nosso
Fundo / cartão	`#F7F7F7` / `#FFFFFF`	nosso
Raios	8 · 12 · 16 · pílula	nosso
Fonte	Inter, system-ui	livre

Detalhes e fontes: `docs/design-ifood.md` §5 e `docs/racional-ux.md` §6–7.

Telas

Em grupo



1 · Início. Home no padrão do app: "Entregar em", busca, verticais com *Match* (selo "novo"), banner "Fazer um match" e aba central Match. Itens fora do protótipo avisam em vez de parecer quebrados.



2 · Quem vai comer? Folha inferior com duas portas: *Só eu* (a IA acha o pedido em segundos) ou *Em grupo* (a turma entra por link ou QR, sem login).



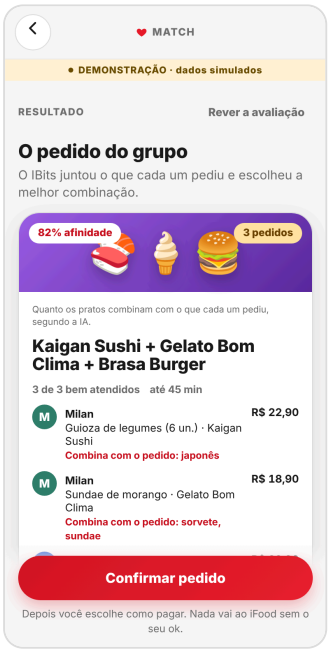
3 · Sala do organizador. QR e link com o IP da rede (os celulares abrem direto); lista "quem entrou" com status em palavra; "3 de 3 convidados responderam"; botão Rodar Match fixo no pé.



4 · Convidado. Sem cadastro: nome, uma frase do seu jeito, orçamento opcional e chips de preferência e restrição, com aviso honesto sobre alergias.



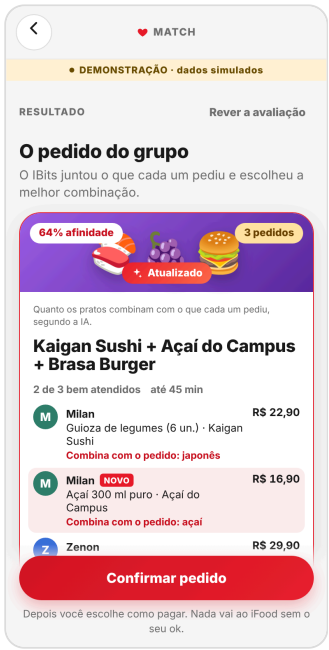
5 · Como o IBits avaliou cada loja. Um avaliador por loja, que só lê o cardápio dela; preço e prazo vêm do MCP; contador de chamadas ao MCP; botão Pular e, no fim, "Ver o pedido".



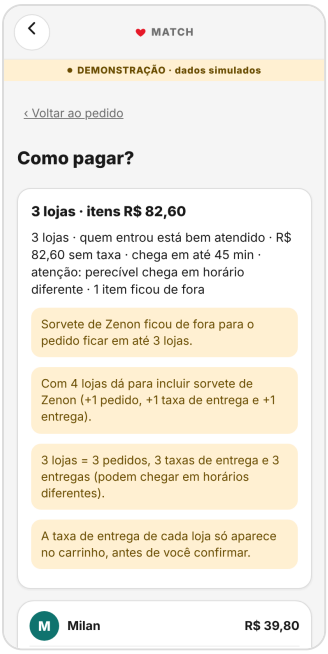
6 · O pedido do grupo. O IBits já escolheu a melhor combinação e ela aparece direto, num cartão só: lojas, item de cada pessoa com o motivo, total só de itens ("taxa no carrinho"), número de entregas; afinidade explicada em uma linha. Mudar = "Ajustar com o IBits"; "Confirmar pedido" fica fixo embaixo. O convidado vê só a parte dele.



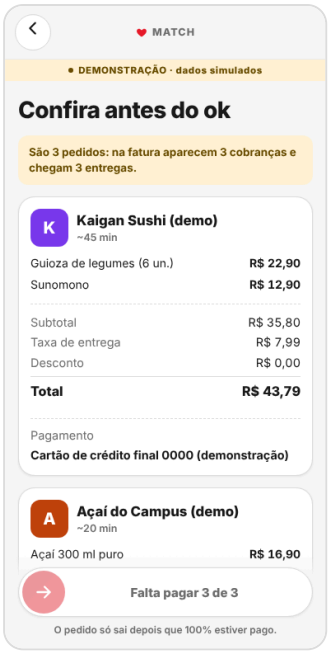
7 · Ajustar com o IBits. Conversa depois do resultado: "troca o sorvete do Milan por açaí" → "Entendi: Milan: açaí no lugar do sorvete. Pronto, atualizei o pedido.". Sugestões geradas a partir de quem respondeu.



8 · Pedido atualizado. O Match roda de novo só com a mudança; o cartão ganha o selo "Atualizado" e o item novo, "NOVO". Os convidados veem "O organizador ajustou o pedido".



9 · Como pagar? Depois de "Confirmar pedido": resumo do pedido com os avisos honestos (ex.: o item que ficou de fora para caber em 3 lojas) e a escolha entre pagar tudo de uma vez ou a vaquinha.



10 · Confira antes do ok. "São 3 pedidos: 3 cobranças e 3 entregas"; carrinho por loja com subtotal, taxa e total lidos do carrinho; forma de pagamento de cada carrinho.



11 · Vaquinha. Cada um paga a sua parte (taxa dividida só entre quem pediu da loja); Pix direto para o organizador com aviso "Demonstração — não pague"; prazo visível; o pedido só sai com 100% pago.

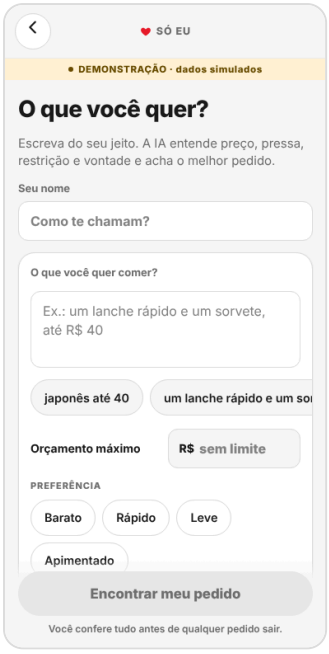


12 · Confirmar envio. "Deslize para enviar" (com alternativa por botão) abre a folha de resumo; nada é enviado sem "Confirmo enviar".



13 · Pedidos. Um cartão por loja com a linha do tempo do acompanhamento (confirmado → em preparo → saiu → entregue), vinda do `trackOrder`.

Só eu



14 · Só eu. Uma tela: "O que você quer?" com exemplos para tocar, orçamento e chips; "Encontrar meu pedido".



15 · Só eu: confira. Sem vaquinha nem QR: "É 1 pedido: 1 cobrança e 1 entrega", carrinho e "Deslize para enviar".

05

Documento Técnico Resumido

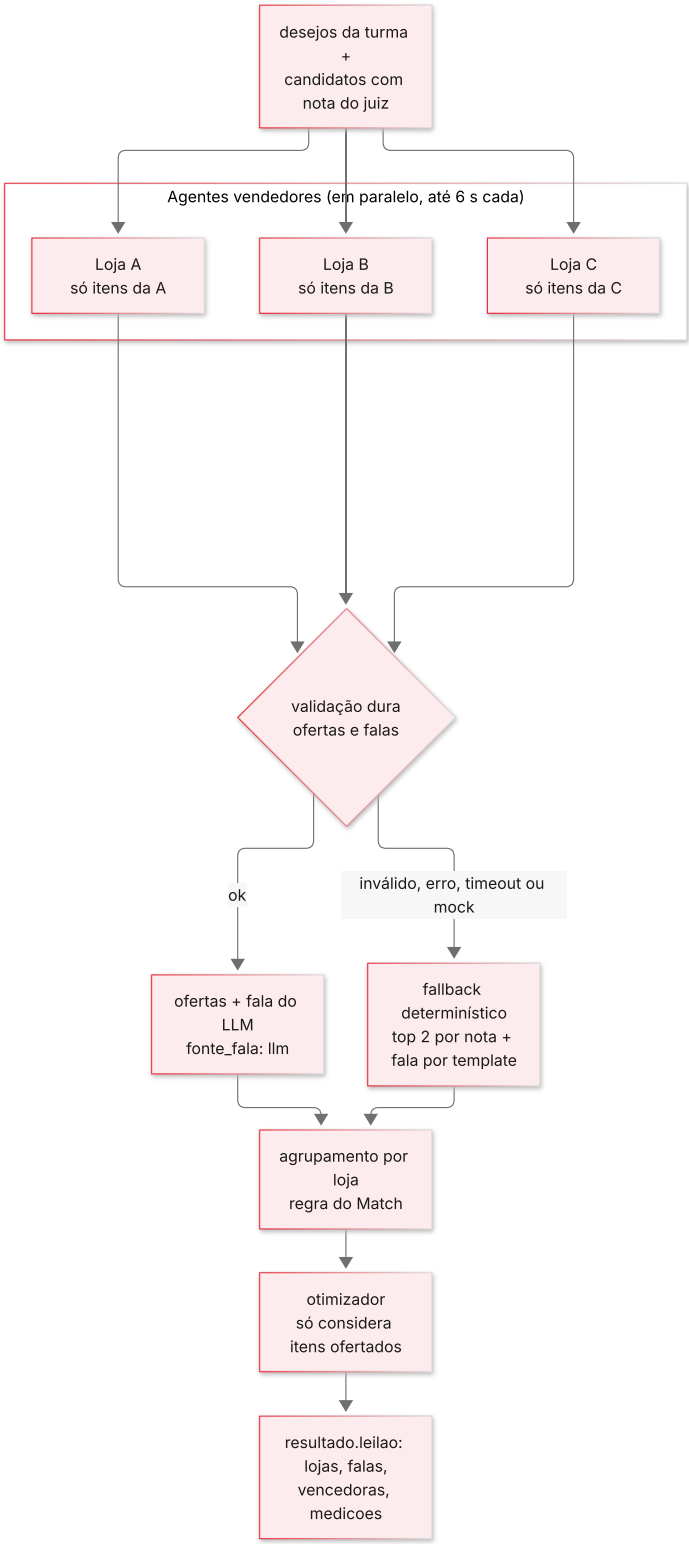
Fonte no repositório: docs/tecnico.md §5–12

> Continuação de `docs/tecnico.md` (seções 5 a 12). Visão geral, componentes, sequência e fluxo do motor estão no entregável 02.

5. Leilão de agentes por loja

Contratos C11 e C12 de `docs/planos/etapa-04-leilao-e-entrega.md`. Decisões 9 e 10 de `docs/decisions.md`. Código: `match/leilao.py` (chamado pelo `motor.py`).

Ideia: o tema é "Vendendo com Agentes". Cada restaurante tem um **agente vendedor** que só enxerga o próprio cardápio (vindo do MCP) e disputa o pedido; o **IBits**, agente comprador do grupo, decide entre os lances.



Quem decide o quê:

- **Agente da loja (LLM):** o que oferecer a quem (até 2 itens por slot, entre os candidatos válidos da própria loja), se passa a vez, e **como dizer** (fala e réplica). Saída JSON: `{"ofertas": [{"slot_id", "item_id", "motivo"}], "passo": bool, "fala": str, "replica": str}`.
- **Regra do Match (código):** agrupar no mesmo pedido os itens de uma loja que atende 2 ou mais pessoas ("1 entrega em vez de 2"). A decisão é do código; só o texto da réplica pode vir do agente, se passar na validação.

- **IBits (código):** abertura, fechamento, vencedoras (lojas da 1ª combinação do otimizador) e a frase "a melhor da sala", calculada só quando uma loja tem a maior avaliação sem empate.

Como funciona:

- Entram no leilão até 8 lojas que aparecem nos candidatos: a melhor loja de cada slot (ninguém perde a única opção) e o resto por melhor nota do juiz. Um agente por loja com algo válido, em paralelo (`asyncio.gather`), com tempo máximo de 6 s cada; loja sem item válido passa sem gastar chamada.
- Entrada do agente: só os itens daquela loja por slot (nome, descrição, categoria, tags, preço do MCP, preço "de" se houver promoção, nota do juiz), o texto de cada pessoa, ETA e avaliação da loja. Os textos são tratados como dados (o prompt manda ignorar instruções dentro deles).
- **Validação das ofertas:** `slot_id` existe; `item_id` é candidato válido daquela loja naquele slot (sem violar restrição nem orçamento); no máximo 2 por slot; motivo ≤ 90 caracteres, sem número e honesto (senão fica o motivo do juiz). Oferta inválida é descartada.
- **Validação das falas e da réplica** (≤ 200 caracteres). A fala do LLM é trocada pelo template (`fonte_fala: "modelo"`) se tiver:
  - **número** (em algarismo ou **por extenso**, como "dezoito e noventa") fora do conjunto permitido: preços ofertados, preço "de" da promoção, ETA, avaliação, contagens, afinidade e números do nome/descrição do item;
  - **contagem** que não bate ("fecho 3 sorvetes" só com 3 sorvetes ofertados; "atendo 3 pessoas" só se atende 3);
  - **"todo mundo"/"todos"** sem atender todas as pessoas;
  - **restrição ou alegação de saúde** (vegano, vegetariano, sem lactose, sem glúten, light, fit, orgânico, saudável...) que nenhuma **tag** dos itens ofertados confirma (o nome do próprio item conta como dado do MCP);
  - **promessa** (grátis, frete, taxa, cupom, brinde, cashback; "promoção"/"desconto" só se o item tiver preço "de");
  - **superlativo ou comparativo** ("o melhor", "o mais barato"): o agente não vê as outras lojas.
- **Fallback** (`LLM_PROVIDER=mock`, erro, timeout, JSON inválido ou nenhuma oferta válida sem "passo"): top 2 por nota do juiz em cada slot da loja; fala por template a partir dos números. Rede de segurança: um slot que tinha opção válida nunca fica sem oferta (se todos os agentes o esqueceram, entra o top 2 por template).
- **Garantia:** com `LLM_PROVIDER=mock` o resultado do roteiro da demo não muda (`test_leilao.py`).
- **Roteiro exibido:** abertura do IBits, até 2 passes, até 5 lances (o Patrocinado primeiro, o mais forte por último; toda vencedora tem o lance mostrado), a réplica da vencedora que agrupa mais pessoas e o fechamento do IBits ("N de M bem atendidos"; com mais de 1 loja, "São N pedidos, e as taxas aparecem no carrinho antes do seu ok"). A taxa de entrega **não** aparece nas falas: ela só existe no carrinho.
- **Medições** no bloco `leilao.medicoes`: `ms_total`, `ms_por_loja` e `chamadas_llm` (só conta resposta JSON válida). `leilao.llm` só é `true` se ao menos uma resposta do LLM foi usada. Na tela, cada fala diz a origem: "agente da loja (IA)" ou "texto conferido".
- **Desligar:** `MATCH_LEILAO=off` no `.env` volta ao motor sem leilão.

Patrocinado = só visibilidade. A loja patrocinada tem o lance **mostrado primeiro** entre os lances e o rótulo "Patrocinado" (em texto). Não garante vaga entre as lojas consultadas no leilão e **nunca** muda nota, filtro ou escolha do otimizador. No demo, 1 loja patrocinada (`loja-demo-poke`) que não vence: mostra que pagar não compra o pedido. No modo real nenhuma loja é patrocinada. Configuração em `match/politica_lojas.json`.

No front:

- `telas/Leilao.tsx` — "A disputa pelo seu pedido": falas uma a uma (~0,8 s cada, até 6 na tela; se o roteiro for maior, ficam abertura, vencedoras e patrocinada, 1 réplica e o fechamento, sem mudar texto nem número), botão **Pular** sempre visível, legenda do Patrocinado, origem de cada fala e o tempo da disputa (`medicoes`). No aparelho do organizador a tela não avança sozinha (ele narra); no do convidado avança cerca de 4 s depois do fechamento.

- `telas/Deck.tsx` — até 3 combinações em cartões: afinidade, "N de M bem atendidos", lojas e, por pessoa, item · loja · preço · motivo. Deslizar ou botões equivalentes ("Ver próxima" / "Dar match"); depois "É um Match!" (`componentes/leilao/EhUmMatch.tsx`, com confete que respeita "reduzir movimento"). Convidado vê o deck só leitura.

6. Otimizador

Código puro em `match/otimizador.py`. A IA só fornece a nota de cada item e o perfil de cada pessoa.

```
S(C) = média_p(sat_p) + α·mín_p(sat_p) − β·(custo_total / orçamento_ref) − γ·(atraso_max / 60)
       − δ·(lojas − 1) − ε·penal_perecível − penal_slots_fora
```

- `sat_p` = média das notas dos slots escolhidos da pessoa `p` (0 se ela ficou de fora).
- **Justiça:** o termo `α·mín` puxa para cima quem ficou pior; ninguém fica com nota baixa para os outros ganharem.
- **Pesos por perfil** (`barato`, `rapido`, `qualidade`, `equilibrado`); com vários perfis no grupo, média campo a campo. Padrão: α 0,5 · β 0,3 · γ 0,2 · δ 0,15 · ε 0,3. "Barato" sobe β para 0,9; "rápido" sobe γ para 0,6.
- **Restrições duras:** item que viola restrição sai; o orçamento da pessoa vale para a **soma** dos slots dela.
- **Slots por pessoa:** o 2º item em diante é opcional (penalidade 0,35 se ficar de fora); o principal paga 1,0.
- **Até 3 lojas** (`MAX_LOJAS = 3` no motor). Se algo ficou de fora, o motor testa 4 lojas e avisa o que entraria ("+1 pedido, +1 taxa, +1 entrega"); a escolha continua com o organizador.
- **Perecível:** ε só se um sorvete/açaí de restaurante vier de loja com ETA diferente em mais de 15 min.
- **Enumeração exaustiva** por conjunto de lojas, com ≤ 8 candidatos por slot, poda acima de 50 mil combinações por conjunto e 120 mil no total. Desempate determinístico (score, depois custo, depois chave estável).
- Taxa de entrega: a busca não traz taxa; o otimizador roda "sem taxa" (`taxa_conhecida: false`) e a tela avisa que a taxa aparece no carrinho, antes do "ok".
- Cada combinação traz os componentes do score e uma explicação curta ("3 lojas · todos com nota ≥ 0,7 · R$ … sem taxa · chega em até … min").

7. Acerto em centavos e vaquinha

`match/acerto.py` (modo "separado"):

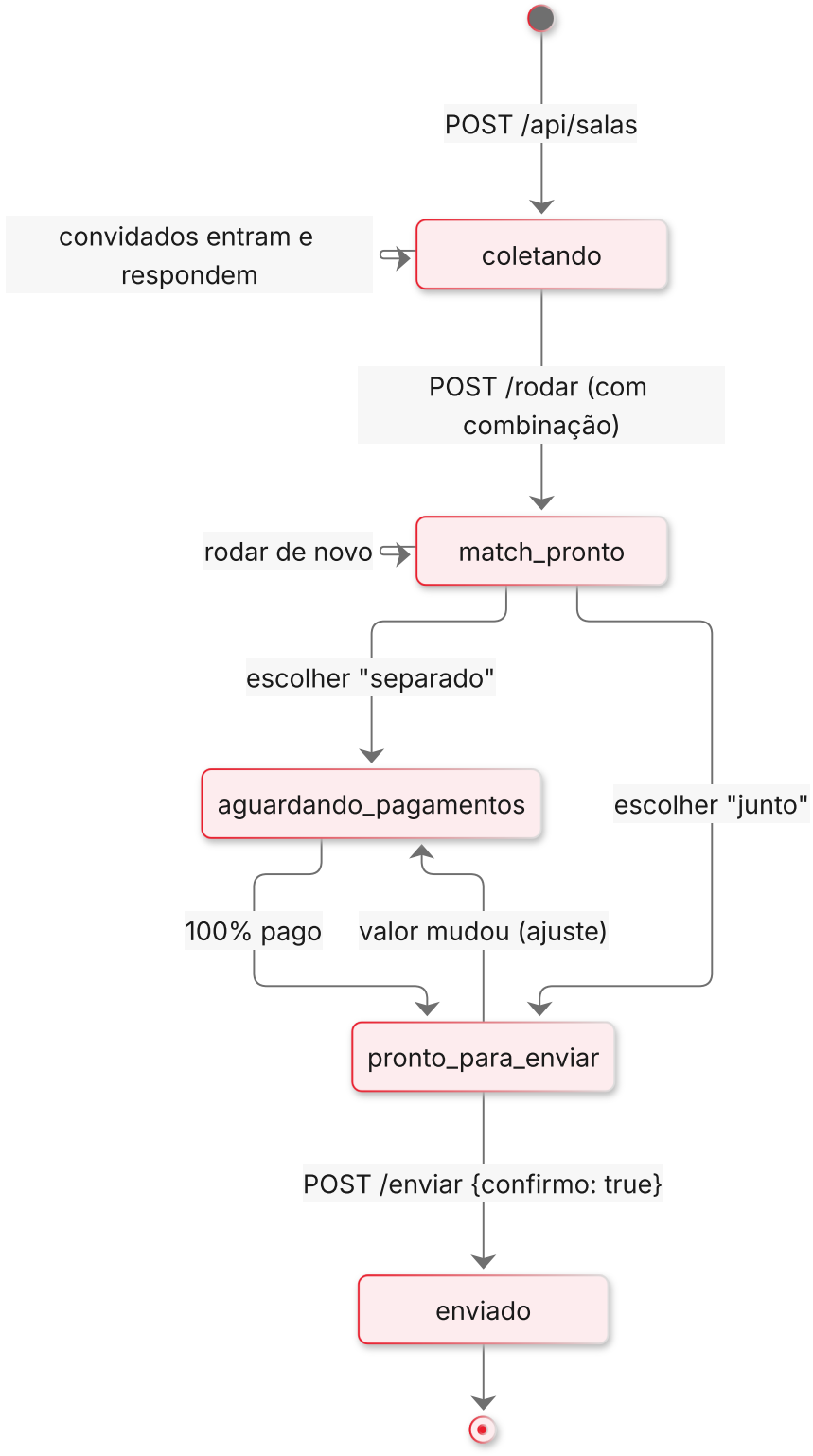
- Cada pessoa paga os próprios itens.
- A **taxa de entrega de uma loja é dividida igualmente entre quem pediu daquela loja**.
- Desconto de uma loja é proporcional ao valor dos itens de cada um nela.
- Tudo em centavos inteiros (`Decimal`, arredondamento meio para cima). A sobra vai pelo método do maior resto, então **a soma das partes é exatamente o total dos carrinhos**. Valores de entrada vêm do `getCart`.
- Prazo para todos pagarem: **10 min + 2 min por pessoa acima de 2**.

Formas de pagamento da vaquinha (o app nunca guarda nem movimenta dinheiro):

- **Pix:** `match/pix.py` gera o "copia e cola" estático do Banco Central (EMV): campos 00, 26 (`br.gov.bcb.pix` + chave), 52, 53 (986), 54 (valor), 58 (BR), 59 (nome ≤ 25), 60 (cidade ≤ 15), 62 (txid) e 63 (CRC16-CCITT-FALSE). A chave é normalizada (CPF/CNPJ só dígitos, telefone +55, e-mail minúsculo, aleatória intacta). O dinheiro vai direto para a chave do organizador; ele marca "recebi".
- **Cartão simulado:** só no modo `demo`, com selo. No modo real a API recusa (409).
- **Dinheiro:** o organizador marca "recebi".
- Pendências: o organizador pode cobrir a parte de alguém ou remover os itens dessa pessoa (recalcula carrinho e partes).

No modo "junto" não há partes: a sala vai direto para "pronto para enviar".

8. Máquina de estados da sala



Regras: rodar sem nenhuma combinação devolve 409 e a sala fica em `coletando`. Transição fora da tabela é recusada. Máximo de 6 participantes (limite do otimizador). Envio parcial (falha no 2º pedido) guarda os já enviados em `ja_enviados` e nunca os reenvia; tentar de novo continua de onde parou.

9. Segurança e confiança

- **Consentimento:** `submitOrder` só é chamado por `checkout.enviar_pedidos(..., consentimento=True)`, que a API só passa com `{"confirmo": true}` do organizador (token no header), no estado `pronto_para_enviar` (100% pago no modo separado). O servidor de demonstração também recusa `submitOrder` sem `userConsent: true`.
- **Preço só de tool:** preço, taxa, prazo e disponibilidade vêm da resposta do MCP. Antes de cada envio o `getCart` é chamado de novo; se o total mudou, nada é enviado, o carrinho é remontado e o organizador confirma de novo. Preço de item diferente entre a busca e o carrinho gera aviso.
- **IA sem números:** o LLM interpreta e dá nota; a conta é código. No leilão, fala com número fora do lance (algarismo ou por extenso), contagem errada, restrição sem tag, promessa ou superlativo é trocada pelo template (seção 5).
- **Tokens e segredos fora do git:** client id/secret só no `.env`; tokens em `backend/.tokens_ifood.json` (ambos no `.gitignore`), gravados com troca atômica. Segredos com `repr=False` na configuração. Token, refresh, secret e code nunca vão para log, exceção nem resposta HTTP. Os testes zeram as credenciais e isolam o arquivo de tokens.
- **OAuth:** PKCE S256, `state` válido por 10 min, `resource=` (RFC 8707). Erros de login viram 401 com mensagem legível.
- **Sanitização:** o diagnóstico mascara nome, e-mail, telefone, CPF, meios de pagamento, endereço textual (coordenadas arredondadas) e qualquer campo com "token" antes de gravar; erro sem traceback cru. O rastro omite localização e dados pessoais. O `GET` da sala não devolve o token do organizador, a chave Pix inteira (só mascarada), o texto do desejo dos outros nem o endereço completo.
- **Trava de envio real:** no modo real, `ClienteIFood.chamar("submitOrder")` recusa a chamada antes de ir à rede, salvo `IFOOD_MCP_PERMITIR_ENVIO=true` no `.env` (padrão `false`). A recusa entra no rastro como erro e a API devolve **409** legível ("Conta de teste: o envio real está desligado (IFOOD_MCP_PERMITIR_ENVIO). Os carrinhos ficaram prontos no iFood."). `GET /api/saude` expõe `envio_real_permitido` (sempre `true` no modo demo) e `ip_local` (IP da rede local para o QR dos convidados). No modo demo o envio segue liberado, com `userConsent: true` exigido pelo servidor de demonstração.
- **Nunca pagar teste:** o Pix de um pedido da conta de teste não tem reembolso. O diagnóstico nunca chama `submitOrder`; a demo real para no `getCart` por causa da trava acima.
- **Selo:** todo dado simulado aparece com o selo DEMONSTRAÇÃO; no modo real, "iFood real · conta de teste".

10. Uso correto do MCP

O que a documentação pede	Como cumprimos
`listAddresses` primeiro	O motor chama `listAddresses` antes de qualquer busca; sem endereço, para com aviso.
`query` só descritivo; preço/distância em `refinements`	A busca manda o termo descritivo ("sushi", "sorvete"), `resultKind`, `limit` e as coordenadas do endereço; orçamento e restrições são filtrados no nosso código, não na `query`.
1 carrinho = 1 restaurante + 1 endereço	`checkout.py` agrupa a combinação por loja e monta 1 carrinho por loja.
`createCart` não é idempotente	Uma trava por sala serializa as ações do organizador que falam com o MCP; a máquina de estados só aceita "escolher" em `match_pronto`, então clique duplo não cria carrinho em dobro.
Não confundir `itemId`, `cartItemId`, `cartId`	O checkout guarda o dono de cada `cartItemId` e confere o `getCart` contra o que foi adicionado.
Método de entrega e pagamento vêm das tools	Usa o `recommendedMethodId` de `agentHints` se estiver na lista; senão o 1º. Pagamento: o id pedido, senão o 1º crédito, senão o 1º (com aviso). Nunca inventa id.
`getCart` para conferir o total	Valores da tela vêm do `getCart`; ele é chamado de novo antes de cada `submitOrder`.

O que a documentação pede	Como cumprimos
`submitOrder` com `userConsent: true` só após confirmação humana	Seção 9.
`trackOrder`: seguir `agentHints.retryMessage` e parar em `pollingStop`	`checkout.acompanhar` devolve `proxima_consulta` (retryMessage) e `encerrado` (pollingStop). A consulta é sob demanda (botão "Atualizar"), sem polling automático.
Nomes das tools	O demo usa os nomes da doc; o diagnóstico grava o `list_tools` real para confirmar. Diferenças entre doc e real vão para `docs/feedback-mcp.md`.

| (não está na doc) O MCP real exige `userLocation` (objeto) na busca | O cliente lê o `input_schema` de cada tool e converte `latitude`/`longitude` em `userLocation`; argumentos que a tool não declara são descartados e aparecem no rastro. |

Formato real das respostas: só `busca.py` e `checkout.py` leem campos das tools. A leitura de endereço já aceita formatos variados; o ajuste fino dos demais campos ao MCP real **[pendente]** depende das amostras do diagnóstico (`docs/mcp-real/`), gravadas no PC da equipe.

11. Limites conhecidos e próximos passos

Limites hoje:

- Busca e checkout seguem o formato da documentação (com endereço e `userLocation` já adaptados ao real); o ajuste dos demais campos depende das amostras do diagnóstico rodado no PC da equipe. `getItem` (opções e preço promocional) ainda não é chamado pelo motor.
- No modo real, `submitOrder` fica travado por padrão (`IFOOD_MCP_PERMITIR_ENVIO=false`): a demo real termina com os carrinhos prontos, sem pedido enviado.
- Com `LLM_PROVIDER=mock` as falas das lojas são sempre por template; o tempo do leilão com IA real ainda não foi medido (o `medicoes` existe para isso).
- Não há tool de cancelamento no MCP: se o 2º pedido falhar depois do 1º enviado, o 1º fica. O app envia em sequência, para no 1º problema e mostra o que saiu.
- O MCP não junta pedidos de lojas diferentes: N lojas = N pedidos, N taxas, N entregas (o app avisa antes).
- Vouchers (`listVouchers`/`setVoucher`), `scheduledTime` e endereço novo (`createAddress`) não são usados.
- Até 6 pessoas por sala; o login OAuth só volta para o `localhost` do PC que roda o app.

Próximos passos:

- Ajustar `busca.py`/`checkout.py` às amostras reais; rodar o Match real até o `getCart` com taxa real.
- Ajustar prompts com IA real e medir o tempo do leilão com `medicoes` (meta ≤ 8 s).
- Depois do hackathon: `getItem` para opções do item, vouchers por loja, `scheduledTime` para alinhar perecíveis, memória do grupo e histórico (`getRecommendations`).

12. Testes

- **306 testes** no backend (pytest), todos passando em 18/10/2026 (contagem desta versão; `./iniciar.sh --testes` mostra o número exato): `cd backend && .venv/bin/python -m pytest -q`. Typecheck do frontend: `cd frontend && npm run typecheck`. Tudo junto: `./iniciar.sh --testes` (Windows: `iniciar.ps1 -Testes`).
- O que cobrem (arquivos em `backend/tests/`):
  - `test_ifood_mcp.py`: a sequência obrigatória inteira via cliente MCP; `submitOrder` sem consentimento falha; rastro de cada tool.
  - `test_otimizador.py`, `test_acerto_pix.py`: restrição elimina; justiça; perfil muda o vencedor; partes somam o total ao centavo; prazo; CRC e campos do Pix; normalização da chave.
  - `test_motor.py`: roteiro da demo com `LLM_PROVIDER=mock` trava o top 1; caminho com LLM falso; opção aproximada; "Não achei X" para pedido fora do cardápio.
  - `test_leilao.py`: roteiro mock inalterado com o leilão; agente só enxerga os próprios itens; item de outra loja ou slot é rejeitado; mais de 2 por slot é cortado; fala com número inventado, por extenso, contagem

errada, promessa, restrição sem tag ou superlativo cai no template; preço "de" da promoção pode ser citado; agrupamento vira réplica; passe do agente e rede de segurança; agente lento usa o fallback sem derrubar o leilão; `chamadas_llm` só conta JSON válido; item cortado conta 0 na afinidade; "a melhor da sala" só sem empate; Patrocinado não muda a combinação; `MATCH_LEILAO=off`.

- `test_interprete.py`: consulta livre sem IA ("coxinha", "feijoada completa"), alternativas, restrição fora da consulta, quantidade repete o slot.
- `test_checkout.py`: sem consentimento não envia; total mudou para; falha no 2º pedido mantém o 1º.
- `test_salas.py`: fluxo inteiro da API; sem token → 403; enviar antes de 100% pago → 409; envio real desligado → 409 legível.
- `test_oauth.py`, `test_diagnostico.py`: PKCE S256, `state` inválido, troca e renovação, token nunca em resposta ou log, 401 → renova 1 vez; sanitização com payloads cheios de dados pessoais.
- `test_endereco_localizacao.py`: endereço em formatos variados e preferência pelo último usado; `userLocation` pelo `input_schema`; argumentos descartados no rastro; `submitOrder` real travado por padrão.
- `test_saude.py`: `/api/saude` com `ip_local` para o QR e `envio_real_permitido`.

06

Feedback sobre os MCPs

Fonte no repositório: docs/feedback-mcp.md

Top 4 por gravidade

1. **Conta de teste não consegue salvar endereço (`ADR-2001`).** `createAddress` geocodifica (`SINGLE`), mas salvar falha sempre com `ADDRESS_CREATION_FAILED` / `ADR-2001` ("unexpected status: 400 Bad Request") nas **3 contas de teste**, inclusive repetindo o exemplo exato da doc do portal. O erro não está documentado e vem com `retryable: true`, mas nunca passa. Sem endereço não há `createCart`, ou seja, nenhum pedido. Pedimos ajuda aos mentores.
2. **`userLocation` exigido e sem documentação.** `searchOffers` e `getRecommendations` recusam `latitude`/`longitude` soltos (como nos exemplos da doc) e pedem `userLocation: {latitude, longitude}`. No 1º teste real nenhuma busca funcionou. Contornamos lendo o `input_schema` do `list_tools` e montando o objeto (`backend/app/ifood_mcp/cliente.py`).
3. **23 tools no servidor × 24 na documentação.** `listItemTaxonomy` está na doc e não existe no `list_tools` real.
4. **`listAddresses` com formato diferente do exemplo da doc.** Nosso leitor não achou a lista e a sala dizia "nenhum endereço salvo", sem erro. Contornamos com um leitor que aceita formatos variados (`match/busca.py`).

O que funcionou bem

- **OAuth com PKCE S256 conforme os metadados** de `id.ifood.com.br`: seguindo os metadados (não a doc), o login da conta de teste funcionou.
- **Mensagem de erro do `userLocation` clara:** diz o campo, o tipo (objeto) e os subcampos numéricos; deu para corrigir sem adivinhar.
- **`list_tools` com `input_schema` útil:** foi o que permitiu adaptar os argumentos ao servidor real e descartar o que a tool não declara.

Amostras sanitizadas do MCP real (`list_tools` e respostas de leitura, geradas por `app.ifood_mcp.diagnostico` no PC da equipe): `docs/mcp-real/` — enviadas pela equipe; se a pasta não estiver no repositório, ainda não foram publicadas.

Registro

Anotar **na hora** cada dor real: data/hora, o que tentamos, o que aconteceu, impacto, sugestão.

Quando	Situação	O que aconteceu	Impacto	Sugestão
17/10 manhã	`POST /marketplace` sem token	401 diz "Your client should automatically re-register and obtain new tokens", mas o `id.ifood.com.br` não publica `registration_endpoint` (sem registro dinâmico de cliente)	Clientes MCP que seguem a mensagem tentam DCR e falham; confunde quem está começando	Ajustar a mensagem ("use o client_id/secret do MCP Portal") ou oferecer DCR
17/10 manhã	Documentação (llms-full.txt, seção de endpoints e quickstart)	Cita `auth.ifood.com`, `dev-auth.ifood.com`, `api-mcp.ifood.com/v3/*`, `mcp.ifood.dev` e `npm install @ifood/mcp-sdk` — nenhum resolve / o pacote não existe no npm. O real é `id.ifood.com.br` + `consumer-mcp.ifood.com.br/marketplace`	Quem segue a doc perde tempo com URLs mortas	Atualizar os exemplos com os endpoints reais

Quando	Situação	O que aconteceu	Impacto	Sugestão
17/10 manhã	Doc recomenda "Pending Action" (QR/device code) para agentes	`/device` e o grant `device_code` não aparecem em `id.ifood.com.br/.well-known/openid-configuration`	Não dá para saber se o fluxo recomendado para agentes existe	Publicar o grant nos metadados ou marcar como "em breve"
17/10 manhã	Doc da Web Login	Diz `code_challenge_method: pkce` e scopes "comma-separated"; os metadados reais pedem `S256` e o padrão OAuth separa scopes por espaço	Implementação literal da doc falha	Corrigir para `S256` e espaço
17/10 manhã	Cadastro do agente	O vídeo dizia "uma Redirect URL por agente", mas o formulário já aceita várias (ok). Escopos continuam **imutáveis** depois de ativo	Errar o escopo obriga a recriar o agente e a passar de novo pela homologação	Permitir editar escopos com nova revisão; atualizar o vídeo
17/10 noite	Doc de contas de teste	O link "criação de conta" aponta para `mcp-developer.ifood-sandbox.com.br/pt-BR/sign-up`, domínio que não respondeu no nosso teste; o portal real é `mcp-developer.ifood.com.br` (botão "Faça login")	Equipe aprovada não achava onde entrar no portal	Corrigir o link e pôr um botão "Meus agentes" visível na doc
17/10 ~22h	`list_tools` no MCP real (conta de teste)	O servidor expõe **23 tools**; a doc lista 24 — `listItemTaxonomy` não existe	Quem planeja pela doc conta com uma tool que não há	Atualizar a lista da doc ou publicar a tool
17/10 ~22h	`searchOffers` e `getRecommendations` com `latitude` / `longitude` soltos (como os exemplos da doc)	Erro: "`userLocation` is required and must be a JSON object with numeric `latitude` and `longitude`" — 0 resultados	Nenhuma busca funcionou no 1º teste; o agente não encontra lojas	Citar `userLocation` (objeto) na doc e nos exemplos; a mensagem de erro é boa e deveria ser o padrão das outras tools
17/10 ~22h	`listAddresses` no MCP real	Formato diferente do exemplo da doc (nosso leitor não achou a lista e a sala dizia "nenhum endereço salvo")	Integração quebra em silêncio	Publicar o `outputSchema` das tools (o `list_tools` já tem o campo) e manter a doc igual ao real

| 18/10 00h48 | `createCart` com `merchantId` + `addressId` (o schema não marca nada como obrigatório) | Volta `MISSING_REQUIRED_FIELDS`: "addressId, merchantId, and items are required when orderId is absent". O `input_schema` tem `required: []` | Sequência natural "criar carrinho → adicionar itens" (a da doc) falha; o schema não avisa | Marcar no schema (ou num `oneOf` com/sem `orderId`) os campos realmente exigidos, ou aceitar carrinho vazio | | 18/10 00h48 | Erro de negócio do `createCart` | Vem como **resposta normal** (`{"error": {"code", "message", "userFriendlyMessage"}}`, sem `isError`) | Cliente MCP genérico (e o nosso diagnóstico) trata como sucesso e segue sem `cartId` | Devolver `isError: true` em todo erro de tool (como o `getRecommendations` já faz) | | 18/10 00h58 | Erro do `createAddress` | Outro formato: `{"success": false, "errors": {"mcpCode", "code"}}` (ex.: `ADDRESS_CREATION_FAILED` / `ADR-2001`, "unexpected status: 400 Bad Request", `retryable: true`); o `getRecommendations` usa um 3º (`isError` + texto JSON `{"error": "..."}`) | 3 formatos de erro em 23 tools: cada cliente precisa de 3 leitores | Um envelope de erro único (`isError` + `{code, message, userFriendlyMessage, retryable}`) | | 18/10 01h22 | `createAddress` em 3 contas de teste | `ADR-2001` também em `safe-` e `fair-thales`, repetindo o exemplo exato da doc e com `listAddresses` antes na mesma sessão; `retryable: true` que nunca passa | Bloqueia carrinho e pedido no MCP real (P1/P3/P4 parados) | Documentar o `ADR-2001` (causa e correção); `retryable` só quando repetir resolve; contas de teste com endereço salvo | | 18/10 00h58 | `createAddress` na conta de teste (`unique-thales`) | Geocodifica (`SINGLE`), mas salvar falha sempre com `ADR-2001`, com 2 endereços diferentes, em 1 e 2 passos | Conta de teste sem endereço = nenhum carrinho possível (o `createCart` exige `addressId`) | Contas de teste já com 1 endereço salvo, ou detalhar o ADR-2001 | | 18/10 00h48 | `listPaymentMethods` | O schema pede só `merchantId`

(obrigatório), não `cartId` — a doc o põe depois do `setCartDelivery`, no fluxo do carrinho | Quem passa o `cartId` (natural no fluxo) leva erro; não fica claro se a lista depende do carrinho/endereço | Aceitar `cartId` também ou explicar na descrição que a lista é por loja | | 18/10 00h48 | `getRecommendations` | `contentMode` é obrigatório (enum `RECOMMENDED`, `RECENT_ORDERS`, ...), mas sem `default`; sem ele: "the value provided for `contentMode` ("") is not valid" | Erro em toda chamada "simples" | Default `RECOMMENDED` | | 18/10 madrugada | `searchOffers` / `browseOffers` / `getMerchant` | Nomes diferentes entre tools para o mesmo dado: item `itemName` (busca) × `name` (doc); preço `unitPrice` / `promotionalUnitPrice`; loja aninhada em `merchant{}` no ITEM e na raiz no MERCHANT; o `getMerchant` não traz cardápio, só `featuredItems` (5) e manda usar `browseOffers` com `refinements.merchantId` | Parser escrito pela doc lê vazio (0 candidatos) | Publicar `outputSchema` e manter um formato só de item/loja | | 18/10 madrugada | `browseOffers` | `merchantType` obrigatório e a descrição diz "se o usuário não disse a vertical, pergunte" | Agente de pedido em grupo não tem a quem perguntar no meio da busca; usamos `RESTAURANT` | Permitir omitir (todas as verticais), como no `searchOffers` | | 18/10 madrugada | `getItem` (amostra "Pizzas Doces Grandes") | Item com `optionGroups` de `minSelections: 1` (sabor obrigatório) e `additionalPrice` igual ao `unitPrice` — não dá para saber se o preço final é 75,99 ou 151,98, nem se o `createCart` sem `options` falha | Preço mostrado na busca pode não ser o do carrinho | Dizer na busca se o item tem opção obrigatória (`hasRequiredOptions`) e documentar como `additionalPrice` soma |